MANUAL TÉCNICO Y GUÍA DE ESTUDIO

Sistema Integral de Pasajes y Viajes Interprovinciales

Arquitectura Full-Stack: Angular 21 (SPA) + Flask REST API + SQLite WAL

Proyecto:	Rutas del Perú - Plataforma de Transporte Terrestre
Repositorio Oficial:	https://github.com/BinNorah/Viajes.git
Frontend:	Angular 21 Standalone, Signals, CSS Vanilla modular (sin Tailwind)
Backend:	Python Flask, 100% REST API JSON desacoplado, Sesiones seguras
Base de Datos:	SQLite 3 con modo WAL, transacciones atómicas y restricciones
Roles del Sistema:	Público/Pasajero, Taquillero (POS), Conductor (Chofer), Administrador
Fecha de Generación:	Octubre 2026

Objetivo del Documento: *Servir como material de estudio exhaustivo y de referencia rápida para comprender la arquitectura de software, el diseño de la base de datos, el flujo de llamadas API, la lógica de control de concurrencia en asientos y la implementación de cada módulo en frontend y backend.*

Índice de Contenido

• **1. Arquitectura General y Stack Tecnológico**

• **2. Modelo de Datos y Diseño de Base de Datos (schema.sql)**

• **3. Backend en Flask: Estructura, Seguridad y Endpoints (backend/app.py)**

• **4. Frontend en Angular 21: Estructura de Directorios y Principios de Diseño**

• **5. Módulo Pasajero Público: Búsqueda, Selección de Asientos y Compra con QR**

• **6. Módulo Pasajero Autenticado: Autenticación y 'Mis Viajes'**

• **7. Módulo Taquilla (POS de Ventanilla): Emisión Inmediata y Cobro Físico**

• **8. Módulo Conductor (Chofer): Turnos Asignados y Manifiesto de Abordaje**

• **9. Módulo Administrador: Dashboard Ejecutivo, CRUD de Flota/Rutas y Reportes**

• **10. Algoritmo Crítico: Prevención de Doble Reserva y Bloqueo Temporal (10 min)**

• **11. Resumen de Buenas Prácticas y Preguntas Clave para Examen**

1. Arquitectura General y Stack Tecnológico

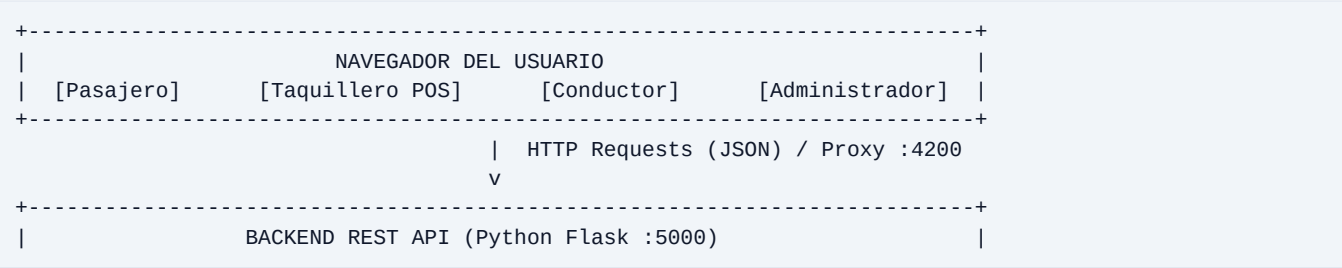
El sistema adopta una arquitectura desacoplada basada en el patrón **Single Page Application (SPA)** en el cliente y un servidor **REST API** en el backend:

• **Frontend (Cliente Web):** Desarrollado con **Angular 21**. Utiliza componentes autónomos (*Standalone Components*), el nuevo paradigma de estado reactivo con **Signals** (signal(), computed()), inyección moderna con inject(), y protección de rutas mediante **Functional Guards** (canActivate: [roleGuard()]).

• **Backend (Servidor de Aplicación):** Implementado en **Python Flask**. Funciona como un servicio 100% REST que intercambia datos exclusivamente en formato **JSON**. No genera HTML en el servidor (cero acoplamiento de plantillas), garantizando velocidad, modularidad y fácil escalabilidad.

• **Persistencia de Datos:** Motor **SQLite 3** operando en modo **WAL (Write-Ahead Logging)** y foreign_keys = ON. WAL permite múltiples lecturas concurrentes sin bloquear escrituras, optimizando el rendimiento en entornos transaccionales.

• **Seguridad y Sesiones:** Control de acceso basado en roles (**RBAC**). Contraseñas hasheadas criptográficamente con algoritmo PBKDF2-SHA256. Cookies de sesión protegidas mediante directivas HttpOnly, SameSite=Lax y tiempo de expiración.



```
|  * Middleware de Sesión (Cookie HttpOnly)                                  |
|  * Decoradores de Seguridad: @need(roles)                                  |
|  * Control de Concurrencia (Bloqueo 10 min de asientos)                    |
|  * Generador de Códigos QR Atómicos y Manifiestos de Pasajeros             |
+----------------------------------------------------------------------------+
                                  |
                                  v  SQL Transactions (PRAGMA WAL)
+----------------------------------------------------------------------------+
|             BASE DE DATOS RELACIONAL (SQLite 3 WAL Mode)                   |
|  terminales | rutas | buses | asientos_plantilla | viajes | boletos | ...|
+----------------------------------------------------------------------------+
```

2. Modelo de Datos y Diseño de Base de Datos

El modelo relacional está normalizado en 3ra Forma Normal (3FN) y definido en backend/schema.sql. Consta de **8 tablas principales**:

Tabla	Descripción y Propósito	Llaves / Restricciones Clave
usuarios	Cuentas del personal y pasajeros. Almacena username, hash de password y rol.	PK: id | UNIQUE(username) | CHECK(rol IN ('ADMIN','TAQUILLERO','CHOFER','PASAJERO'))
terminales	Ciudades y agencias de embarque y desembarque.	PK: id | ciudad, nombre_agencia, direccion
rutas	Trayectos entre terminal origen y destino, distancia y tiempo.	PK: id | FK(origen -> terminales), FK(destino -> terminales)
buses	Flota vehicular, placas, marca, modelo, pisos y capacidad total.	PK: id | UNIQUE(placa) | CHECK(tipo_servicio IN ('CAMA_160','CAMA_180','EJECUTIVO'))
asientos_plantilla	Distribución física de asientos de cada bus (piso, fila, columna, recargo).	PK: id | FK(bus_id -> buses) | UNIQUE(bus_id, numero_asiento)
pasajeros	Datos personales de los viajeros (DNI, nombres, teléfono).	PK: id | UNIQUE(tipo_documento, numero_documento)
programacion_viajes	Salidas programadas: fecha, hora, bus asignado, chofer y estado.	PK: id | FK(ruta_id), FK(bus_id), FK(conductor_id -> usuarios)
boletos_reservas	Transacciones de venta/reserva, asiento ocupado, precio, QR y bloqueo.	PK: id | FK(programacion_viaje_id), FK(asiento_id), FK(pasajero_id)

Restricción Crítica de Integridad: Prevención de Sobreventa

Para evitar físicamente que dos pasajeros compren o bloqueen el mismo asiento en un mismo viaje, la base de datos implementa un **Índice Único Parcial**:

```
CREATE UNIQUE INDEX idx_asiento_viaje_activo
ON boletos_reservas (programacion_viaje_id, asiento_plantilla_id)
WHERE estado_pago IN ('PAGADO', 'BLOQUEADO');
```

> ***¿Cómo funciona?*** *Si un asiento ya está en estado PAGADO o BLOQUEADO, la base de datos arroja un error sqlite3.IntegrityError ante cualquier segundo intento simultáneo. Si un boleto pasa a estado ANULADO, el índice deja de aplicar y el asiento queda automáticamente libre para otro usuario.*

3. Backend en Flask: Arquitectura de app.py

El archivo backend/app.py fue refactorizado y condensado a 840 líneas de código altamente modular, limpio y eficiente. Está estructurado en capas bien definidas:

Funciones Helper de Base de Datos y Utilidades

- conectar(): Abre la conexión a SQLite en modo lectura/escritura (URI con mode=rw), activa llaves foráneas (PRAGMA foreign_keys = ON) y establece busy_timeout = 15000 ms para prevenir bloqueos concurrentes.
- q(sql, params, one=False): Ejecutor de consultas SELECT. Convierte cada fila de tipo sqlite3.Row en un diccionario Python nativo para serialización JSON limpia.
- run(sql, params): Ejecutor de sentencias INSERT, UPDATE, DELETE. Retorna (lastrowid, rowcount) bajo manejo de transacciones con commit automático.
- need(*roles): Decorador de seguridad funcional. Verifica si el usuario actual ha iniciado sesión (sesión en cookie cifrada) y si su rol pertenece al conjunto permitido. Si no cumple, responde automáticamente HTTP 401 (No autorizado) o HTTP 403 (Prohibido).

Ejemplo del Decorador de Seguridad @need:

```
def need(*roles):
    def dec(fn):
        @wraps(fn)
        def wrap(*a, **kw):
            if 'usuario_id' not in session:
                return jsonify({'error': 'No autenticado'}), 401
            if roles and session.get('rol') not in roles:
                return jsonify({'error': 'Acceso no autorizado'}), 403
            return fn(*a, **kw)
        return wrap
    return dec
```

Mapa Completo de Endpoints REST API

Método	Ruta Endpoint	Rol Requerido	Descripción
POST	/api/auth/login	Público	Valida credenciales, inicializa cookie de sesión y retorna rol.
POST	/api/auth/logout	Cualquiera	Destruye la sesión del servidor y limpia la cookie.
GET	/api/auth/yo	Cualquiera	Retorna los datos del usuario conectado en la sesión actual.
GET	/api/publico/ciudades	Público	Lista todas las ciudades con terminales activas para los selectores.
GET	/api/publico/viajes	Público	Búsqueda de viajes programados por origen, destino y fecha.
GET	/api/publico/viajes/<id>/asientos	Público	Retorna la distribución física del bus y el estado de cada asiento.
POST	/api/publico/bloquear-asiento	Público	Bloquea atómicamente un asiento por 10 min mientras completa la compra.
POST	/api/publico/confirmar-compra	Público	Registra pasajero, confirma pago, genera código QR y boleto.
GET	/api/pasajero/mis-viajes	PASAJERO	Historial de pasajes adquiridos por el usuario registrado.
GET	/api/chofer/mis-viajes	CHOFER	Lista de viajes asignados al conductor conectado.
GET	/api/chofer/viajes/<id>/manifiesto	CHOFER / STAFF	Lista de pasajeros y asientos confirmados para el abordaje.
GET	/api/admin/dashboard	ADMIN	Métricas ejecutivas: total ventas, pasajes, buses activos y destinos.
GET/POST /PUT/DEL	/api/admin/crud/<seccion>	ADMIN	CRUD genérico unificado para buses, rutas, terminales y viajes.
GET	/api/admin/reservas	ADMIN	Listado detallado de todas las reservas y boletos emitidos.
GET	/api/admin/reportes	ADMIN	Reporte de ventas agrupado por ruta, fecha y tipo de bus.

4. Frontend en Angular 21: Estructura y Principios

El frontend fue completamente modernizado y estructurado bajo los estándares de **Angular 21**. Cada componente cuenta con su propia carpeta conteniendo sus 3 archivos dedicados (**.ts**, **.html**, **.css**), eliminando código embebido y separando limpiamente la lógica de la presentación.

Organización del Árbol de Carpetas (frontend/src/app/)

```
frontend/src/app/
■
■■■ api.service.ts          <- Servicio HTTP centralizado (GET, POST, PUT, DELETE)
■■■ toast.service.ts        <- Notificaciones flotantes reactivas con Signals
■■■ toasts.component.ts     <- Renderizador visual de alertas y avisos
■■■ guards.ts               <- Functional Guard: roleGuard(['ADMIN', ...])
■■■ app.routes.ts           <- Mapeo de rutas URL a componentes Standalone
■■■ app.component.{ts,html,css} <- Barra de navegación global y contenedor
■
■■■ inicio/                 <- Vista de bienvenida y pasajes directos con precio
■■■ pasajero/
■   ■■■ buscar/             <- Búsqueda con selector de ciudades y fecha
■   ■■■ login/              <- Formulario de autenticación con tabulación de roles
■   ■■■ mis-viajes/         <- Historial de boletos del pasajero autenticado
■■■ mapa/                   <- Visualizador interactivo de asientos (Piso 1 y Piso 2)
■■■ desglose/               <- Resumen de precio: base + recargo piso/ubicación
■■■ pasajero-form/          <- Captura de DNI, nombres, apellidos y teléfono
■■■ boleto/                 <- Comprobante digital con código QR renderizado
■■■ taquilla/               <- Terminal punto de venta (POS) para taquilleros
■■■ chofer/                 <- Panel del conductor con manifiesto de abordaje
■■■ admin/
    ■■■ panel/              <- Layout con sidebar profesional y KPIs ejecutivos
    ■■■ crud/               <- Gestor de tablas: Programación, Buses, Rutas, etc.
    ■■■ reservas/           <- Visualizador de reservas con filtro y estado
    ■■■ reportes/           <- Informes de recaudación y ocupación porcentual
```

Manejo de Estado Reactivo: Signals de Angular

En lugar de depender de librerías externas o ciclos de detección de cambios pesados (Zone.js tradicional), los componentes utilizan **Signals** de Angular. Un *Signal* es un contenedor de valor reactivo que notifica a las partes del DOM que dependen de él cuando el valor cambia, logrando un rendimiento óptimo.

```
// Ejemplo en CrudComponent:
export class CrudComponent implements OnInit {
  seccion = signal('viajes');       // Signal con la sección activa
  registros = signal([]);            // Signal con la lista de filas
  cargando = signal(false);         // Signal para el indicador de carga
  busqueda = signal('');             // Signal para el filtro de texto

  // Computed Signal: se recalcula automáticamente cuando cambian registros o busqueda
  registrosFiltrados = computed(() => {
    const q = this.busqueda().toLowerCase();
    return this.registros().filter(r => JSON.stringify(r).toLowerCase().includes(q));
  });
}
```

5. Módulo Pasajero Público: Flujo de Compra

El flujo de compra para un pasajero público no registrado o registrado consta de 5 etapas fluidas e interactivas:

1. Selección de Ruta y Fecha (Inicio / Buscar): El usuario selecciona la ciudad de origen (ej: Lima) y destino (ej: Arequipa) junto a la fecha deseada. Al pulsar 'Buscar Pasajes' o al hacer clic directo en una de las tarjetas de destinos destacados del Inicio, se cargan los viajes disponibles consultando GET /api/publico/viajes.

2. Mapa de Asientos Interactivo (MapaComponent): Al seleccionar un viaje, se solicita GET /api/publico/viajes/<id>/asientos. Se muestra la planta física del bus dividida por pisos (Piso 1: Asientos Cama 160°/180°, Piso 2: Asientos Ejecutivos). Cada butaca muestra su estado cromático: *Verde* (Disponible), *Rojo* (Ocupado/Pagado), *Naranja* (Bloqueado temporalmente) y *Azul* (Seleccionado actualmente).

3. Bloqueo Temporal (10 Minutos): Al hacer clic en un asiento disponible, se dispara POST /api/publico/bloquear-asiento. El backend genera un registro en boletos_reservas con estado BLOQUEADO y una fecha de expiración calculada a now() + 10 minutos. Esto garantiza que ningún otro usuario pueda reservar el mismo asiento mientras se ingresan los datos del pasajero.

4. Captura de Datos y Desglose (PasajeroFormComponent & DesgloseComponent): Se solicitan DNI/Pasaporte, Nombres, Apellidos y Teléfono. El componente de desglose calcula: Precio Final = Precio Base + Recargo Asiento (por ejemplo, recargo por primer piso o fila delantera).

5. Confirmación y Emisión de Boleto (BoletoComponent): El pasajero hace clic en 'Pagar y Emitir Boleto'. El backend actualiza el estado a PAGADO, genera un código criptográfico único de boleto (ej: QR-VIAJE1-ASIENTO12-71829304) y el frontend renderiza el comprobante listo para imprimir con botón de descarga.

6. Módulos por Rol: Taquilla, Chofer y Admin

6.1. Módulo Taquilla POS (Venta en Ventanilla)

Diseñado para personal de ventanilla en terminales terrestres (TaquillaComponent). Permite:

- **Búsqueda Rápida:** Filtro instantáneo de salidas del día.
- **Cobro en Efectivo/Tarjeta:** Omite pasarelas de pago externas; confirma el boleto directamente en estado PAGADO.
- **Anulación Inmediata:** Capacidad de anular boletos emitidos por error, liberando el asiento al instante para la venta al público.

6.2. Módulo Conductor / Chofer

Diseñado para los conductores en ruta (ChoferComponent). Cuenta con:

- **Mis Turnos Asignados:** Consulta de los viajes en los que el conductor está designado por su ID de usuario.
- **Manifiesto Oficial de Abordaje:** Lista en tiempo real de todos los pasajeros confirmados con su número de asiento, DNI y estado de pago, facilitando el control de embarque en el andén.

6.3. Módulo Administrador

Panel integral de control de operaciones (AdminComponent, CrudComponent, ReservasComponent, ReportesComponent):

- **Sidebar Modular Izquierdo:** Navegación limpia de pantalla completa entre: Dashboard, Programación de Salidas, Flota de Buses, Rutas, Terminales, Reservas y Reportes.
- **Dashboard Ejecutivo:** Tarjetas de estadísticas en tiempo real (Ventas Totales S/, Pasajes Emitidos, Salidas Activas, Destinos Cubiertos) y accesos directos de gestión rápida.
- **CRUD Profesional:** Altas, Bajas y Modificaciones completas sobre todas las entidades. En salidas programadas incluye modal integrado de confirmación de eliminación con protección transaccional.
- **Módulo de Reservas:** Visualización de todos los boletos emitidos en el sistema, permitiendo anulación y reimpresión de comprobantes.
- **Reportes Financieros:** Métricas consolidadas de recaudación por ruta, tasa de ocupación promedio y análisis de demanda.

7. Preguntas Frecuentes y Conceptos para Examen

P: ¿Por qué se desacopló el backend de Flask del frontend de Angular?

R: Porque en arquitecturas modernas el backend debe limitarse a servir APIs REST (datos puros en JSON), mientras que el frontend se encarga de la interfaz de usuario en el navegador (SPA). Esto permite que el backend pueda alimentar simultáneamente aplicaciones móviles, quioscos físicos o integraciones B2B sin modificar una sola línea de lógica.

P: ¿Cómo se evita que dos personas compren el mismo asiento al mismo tiempo?

R: Se aplican dos niveles de defensa: 1) En la base de datos mediante un índice único condicional (CREATE UNIQUE INDEX ... WHERE estado_pago IN ('PAGADO','BLOQUEADO')), lo que genera una excepción de integridad física ante colisiones. 2) En el backend mediante un temporizador atómico de bloqueo de 10 minutos (estado BLOQUEADO).

P: ¿Qué ventaja ofrecen los Signals frente a RxJS o variables normales en Angular 21?

R: Los Signals ofrecen reactividad granular sin sobrecarga de memoria. A diferencia de Zone.js que revisa todo el árbol de componentes ante cualquier evento, un Signal sabe exactamente qué nodo del DOM depende de él y actualiza solo ese elemento de forma síncrona y predecible.

P: ¿Por qué se eliminaron los archivos de prueba y stubs antes del pase a producción?

R: Archivos de test residuales, scripts temporales (como clean_data.py) o archivos planos redundantes ensucian el repositorio, aumentan la superficie de ataque, confunden a los desarrolladores y aumentan el tamaño del despliegue. Un repositorio limpio debe contener únicamente el código fuente de producción y su esquema oficial.

P: ¿Cómo viajan y se aseguran las sesiones entre Angular y Flask?

R: A través de cookies de sesión cifradas con la SECRET_KEY de Flask. Tienen atributos HttpOnly (inaccesibles por JavaScript malicioso desde XSS) y SameSite=Lax (mitigación de ataques CSRF). Los Functional Guards en Angular verifican el rol antes de permitir la navegación.

Fin del Manual y Guía de Estudio · Rutas del Perú © 2026